Applied Soft Computing 52 (2017) 1070–1083

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Comparative study of the use of fuzzy logic in improving particle swarm optimization variants for mathematical functions using co-evolution

Fevrier Valdez, Juan Carlos Vazquez, Patricia Melin*, Oscar Castillo

Tijuana Institute of Technology, Mexico

ARTICLE INFO

Article history:
Received 9 October 2014
Received in revised form 12 July 2016
Accepted 16 September 2016
Available online 21 September 2016

Keywords:
Fuzzy particle swarm optimization
Fuzzy logic
Optimization
Inertia weight adjustment

ABSTRACT

Particle Swarm Optimization (PSO) is a nature-inspired computation technique based on the social behavior of birds flocking or fish schooling, and can be also viewed as a biologically inspired computational search and optimization method. Since its introduction by Kennedy and Eberhart in 1995, several variants of the original Particle Swarm Optimization technique have been created to improve the quality of the solutions, to improve the speed of convergence, and to avoid getting trapped in local minima. The main proposal is to design a new approach to dynamically adapt some parameters of the PSO algorithm, such as the weight of inertia and learning factors. In this regard, fuzzy logic is applied to the design of the proposed approach. To measure performance, a comparison of different approaches for the inertia weight is performed; these approaches are: *constant*, *random adjustments, linear decreasing, nonlinear decreasing and fuzzy adaptive inertia.* The co-evolution concept is also applied to improve the performance of the PSO algorithm for problems of higher degree of complexity. A set of 11 mathematical functions is used to validate the proposed approach. The benchmark functions that are considered are widely used in this field of study.

© 2016 Elsevier B.V. All rights reserved.

1. Introduction

The Particle Swarm Optimization (PSO) is a nature-inspired algorithm that was proposed by Kennedy and Eberhart in 1995 [12,25], and is a stochastic search method based on populations. The idea behind this algorithm is inspired in the social behavior of animals, such as fish schooling or bird flocking. In reviewing recent work on PSO we find that it has advantages and disadvantages, the advantages over other optimization algorithms are its simplicity and easy implementation. PSO does not have many parameters that need to be adjusted; the value of each of these parameters makes possible a quality search for the algorithm. The performance of an optimization algorithm should have the ability of explore different regions and areas of the search space. The speed of the searching is very fast. The calculations in PSO are very simple. The PSO algorithm can be used to solve complex optimization problems. The PSO algorithm has been applied successfully to different problems, such as benchmark function optimization [4,13,22,36], solving permutation problems [15,19] and neural networks [3,21], etc.

The standard PSO algorithm has problems in consistently finding good solutions, such as convergence speed acceleration and finding possible local optima [34], so, there are several modifications that have been proposed to improve the original PSO. The modifications aim at reaching the best solutions faster, increasing the diversity of the swarm of particles and improving convergence of the PSO algorithm. Different works have considered adjusting the PSO parameters. For example, in J. C. Bansal et al. [8] fifteen relatively recent and popular inertia weight strategies are presented. In Zhi-Hui Zhan et al. [39] an adaptive particle swarm optimization that performs the automatic adaptation of acceleration coefficients, inertia weight and other algorithmic parameters is presented. In Yuhui Shi et al. [35] a fuzzy inference system to dynamically control the inertia weight of the PSO algorithm is presented. Husam Shaheenin [6] applied a fuzzy inference system to control the main parameters of PSO, the inertia weight and learning factors c_1 and c_2 during the evolution process. In Oscar Castillo et al. [40] a fuzzy inference system is applied to dynamically control parameters in the Differential Evolution algorithm was presented. In Cinthya Solano-Aragón et al. [41] a fuzzy inference system was applied to control the main parameters of the Firefly algorithm. In

* Corresponding author.
E-mail addresses: epmelin@hafsamx.org, pmelin@tectijuana.mx (P. Melin).

http://dx.doi.org/10.1016/j.asoc.2016.09.024
1568-4946/© 2016 Elsevier B.V. All rights reserved.

Emer Bernal [42] et al. a parameter variation study in the Imperialistic Competitive algorithm was presented. In Maribel Guerrero et al. [43] a fuzzy inference system is applied to dynamically control parameters in the Cuckoo Search Algorithm.

In this work the novelty of the proposal is the design of a new approach to adjust the parameters of the PSO algorithm therefore, unlike the work in [31] a fuzzy inference system is proposed to dynamically control the inertia weight (w) and c_1 and c_2 learning factors. With the proposed method we improved the performance of PSO, because with the adaptation of parameters using fuzzy systems it better than the original method, which only uses static parameters. Usually there are high dimensionality space problems [18], where finding the optimum is easier than on some hard low-dimensional ones [20]. One way to solve these problems with exponential increase in complexity is to use co-evolution [2,20,27]; another one is to divide the search space onto lower dimensional subspaces. In this work co-evolution is applied to functions in high-dimensional space. A fuzzy PSO algorithm is developed to improve the performance of the swarm of particles and the algorithm is compared with basic variations of the PSO algorithm. Mathematical functions are used to measure the performance of the PSO algorithm with their different approaches. The rest of the paper is organized as follows: in Section 2 the basic information for particle swarm optimization algorithm is briefly described, then Section 3 describes the different variants of inertia weight used in this paper, Section 4 explains the structure of our proposed fuzzy system used in this paper, Section 5 reviews the concepts of Co-evolution, Section 6 presents simulation results for the mathematical functions and finally, conclusions are summarized in Sections 7.

2. Particle swarm optimization (PSO)

The PSO algorithm is formed by a set of particles commonly called a swarm, similar to a population in analogy with evolutionary algorithms, where these particles are similar to individuals, and represent potential solutions. In simple terms, while the particles fly through the search space, for each particle, the position is adjusted according to its own experience and the experience of its neighbors. In the search space, the position of particle i at time step t is denoted by $x_i(t)$. Equation (1) defines the changes of the position of the particle by adding a velocity $v_i(t+1)$ to the current position.

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (1)$$

In this behavior, the optimization process is driven by the velocity added in the above equation, where the process reflects both the information from the particle's neighborhood, which is socially exchanged and the experiential knowledge of the particle. Two algorithms such as *gbest* and *lbest* variants of PSO have been developed, where they differ in the size of their neighborhoods. They are described in more detail below.

2.1. Global best PSO

In this case, inside the PSO algorithm, the entire swarm is the neighborhood for each particle. The star topology (refer to in Fig. 2) reflects the social network employed by this algorithm. The best position found in the swarm is referred to as $\hat{y}(t)$. The PSO algorithm is composed of a swarm of particles moving in a j dimensional space. For *gbest* PSO, Equation (2) shows how to calculate the velocity of particle i.

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\hat{y}_j(t) - x_{ij}(t)] \quad (2)$$

Let $v_{ij}(t)$ denote the velocity and $x_{ij}(t)$ the position of particle i in dimension $j(1,\ldots, n_x)$ at time step t, $y_{ij}(t)$ is the personal best position that the particle has visited since the first time step associated with particle i in dimension j, the global best position at time step t, is denoted as $\hat{y}_j(t)$ obtained so far by any particle, c_1 and c_2 are the acceleration coefficients, in which c_1 expresses the contribution of the cognitive component and c_2 expresses the social component, $r_{1j}(t)$ and $r_{2j}(t)$ are random values in the range [0,1], which introduce a stochastic element to the PSO algorithm. Fig. 1 shows the Pseudo code for the *gbest* PSO algorithm that is used.

```
Create and initialize a swarm, S;
Repeat
   for each particle i in S do
      //update the particle's best position
      if f(S.x_ij) < f(S.y_ij) then
         S.y_ij = S.x_ij;
      end
      //update the global best position
      if f(S.y_ij) < f(S.ŷ) then
         S.ŷ_j = S.y_ij;
      end
   end
   //update the particle's velocity and position
   for each particle i in S do
      update the particle's velocity, equation (2);
      update the particle's position, equation (1);
   end
t = t + 1
until stop condition, t > max- iteration;
```

Fig. 1. The gbest PSO algorithm.

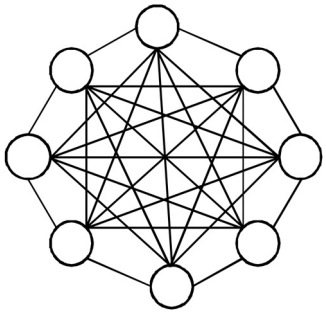


Fig. 2. Representation diagram for Star topology.

Let f ( ) denote a function of j-dimensional space, $S.x_{ij}$ the position of particle i in dimension j, $S.y_{ij}$ the personal best position of the particle i in dimension j and $S.\hat{y}_j$ the global best position in dimension j, where $S.y_{ij} = S.x_{ij} S{\cdot}x_{ij}$ (the personal best position takes the position of the particle), this if $f(S.x_{ij}) < f(S.y_{ij})$ (the value of the function with position $S.x_{ij}$ is less than the value of the function with position $S.y_{ij}$). For example, to set the personal best position the condition is: if $f(S.x_{ij}) < f(S.y_{ij})$ then $S.y_{ij} = S.x_{ij}; S{\cdot}x_{ij}$; when this condition is true, the best personal position is changed by the new particle because was better than the actual best personal position. $f(S.x_{ij})$ represent the actual position of the particle i in the S swarm, and $f(S.y_{ij})$ represent the best global position in the S swarm.

In this topology, all particles are interconnected, where each particle can communicate with every other particle and are attracted towards the best solution. Therefore, each particle imitates the best solution found by the entire swarm. Fig. 2 illustrates the star social structure.

3. Variants of PSO

For achieving a good behavior of the PSO algorithm numerous variants have been developed in other works [1,10,17,35,37], since if the algorithm parameters are appropriately selected, the convergence of the PSO can be guaranteed [23,30,36]. We can note in Equation (3), that the inertia weight (w) is a global variable applied to the whole population, therefore, in this work the inertia weight and acceleration coefficients have been discussed. For controlling the exploration and exploitation abilities and the speed and direction of particles [24], Shi and Eberthart [35] introduced the inertia weight parameter. This parameter, w, basically controls how much memory of the previous flight direction will affect the new velocity.

$$v_{ij}(t+1) = wv_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\hat{y}_j(t) - x_{ij}(t)] \tag{3}$$

The particle velocities decelerate until they reach zero when $w < 1$, the swarm diverges and velocities increase over time when $w \geq 1$. The optimum values for the inertia weight are problem dependent [35].

Approaches to let the inertia weight dynamically vary and can be classified with the following definitions:

- Random adjustments, in each iteration a different inertia weight is randomly selected, as Peng et al., used in [14]
- Linear decreasing, an initially large value is linearly decreased to a small value. From Naka et al. [28], Yoshida et al. [7]

$$w(t) = (w(0) - w(n_t))\frac{(n_t - t)}{n_t} + w(n_t) \tag{4}$$

Let n_t denote the maximum number of time steps for which the algorithm is executed, $w(0)$ and $w(n_t)$ denote the initial and final weight, respectively where $w(0) > w(n_t)$, and the inertia at time step t is denoted by $w(t)$.

- Nonlinear decreasing, an initially large inertia weight decreases nonlinearly to a small inertia weight. The exploration time in nonlinear decreasing methods is shorter than the linear decreasing methods therefore, for smoother search space, nonlinear decreasing methods will be more suitable. In Equation (5) nonlinear methods have been adapted:
  - From Naka et al. [28],

$$w(t+1) = \frac{(w(t) - 0.4)(n_t - t)}{n_t + 0.4} \tag{5}$$

with $w(0) = 0.9$.

- Fuzzy adaptive inertia, the inertia weight values are dynamically adapted with a fuzzy inference system [38]. For the inertia adjustment, Shi and Eberhart defined a fuzzy inference system, which is formed of the following components:
  - The fuzzy system has two input variables such as the current inertia weight (*weight*) and the Current Best Performance Evaluation (CBPE), where this variable measures the efficiency of the best candidate solution found so far by the particle swarm optimization.
  - One output variable which is represented as the change in inertia weight (*w_weight*).
  - In representing membership function as *leftTriangle*, *Triangle* and *rigthTriangle* to the LOW, MEDIUM and HIGH variables, respectively

Table 1
Rules in the Fuzzy adaptive PSO.

Rule Number	Input		Output
	CBPE	weight	w_weight
1	1	1	2
2	1	2	1
3	1	3	1
4	2	1	3
5	2	2	2
6	2	3	1
7	3	1	3
8	3	2	2
9	3	3	1

  - Table 1 shows the nine rules from the fuzzy inference system for the change in inertia weight. Following the three variable definitions (LOW, MEDIUM and HIGH) we can find the representation of the rules in the fuzzy inference system where the number **1** encodes fuzzy set LOW, **2** encodes MEDIUM and **3** encodes HIGH. For example **"112"** represents a fuzzy rule that is [38]:

if *NCBPE* is LOW, and
Weight is LOW
Then *w_weight* is MEDIUM

The Normalized Current Best Performance Evaluation (NCBPE) is represented as:

$$NCBPE = \frac{CBPE - CBPE_{\min}}{CBPE_{\max} - CBPE_{\min}} \tag{6}$$

Let $CBPE_{min}$ denote the estimated minimum and $CBPE_{max}$ the non-optimal CBPE for which represents that any solution with CBPE greater or equal to $CBPE_{max}$ is not a viable solution to the minimization problem.

4. Proposed fuzzy system for dynamic parameter adjustment in PSO

From the velocity Equation (3) of the PSO algorithm, the inertia weight parameter is very important to optimally balance exploration and exploitation and to ensure convergence behavior [26]. For the inertia weight, large values promote easy exploration with increased diversity and small values, local exploitation is promoted. The stochastic influence of the cognitive and social components on the total velocity of a particle is controlled by the learning factors c_1 and c_2 [26], where we express how much confidence a particle has in itself and in its neighbors, respectively. For c_1 and c_2, high values inflict sudden movements, causing more acceleration towards good regions and low values allow good regions to be explored.

According to the above information a fuzzy model is proposed to obtain the optimal values for the PSO algorithm, where the inertia weight w and the learning factors c_1 and c_2 are dynamically adapted through the use of fuzzy sets and rules during the iterative process [5,32,38].

A system structure with, a two-input, three-output and nine fuzzy rules, has been proposed. The two inputs are the number of iterations with an unchanged best fitness (*NU*), and the current value of the inertia weight (w). The three outputs to obtain new values for c_1, c_2 and *chw* (change of inertia weight).

Fig. 3 illustrates the proposed fuzzy model. This model has 2 inputs, which are the number of iterations with unchanged best fitness (*NU*) (for each iteration, if the best fitness has not changed the variable is increased, once the fitness takes a better value than the previous one, the variable is reinitialized to zero) and current inertia weight (w), and 3 outputs which are the new values for acceleration factors (c_1 and c_2) and the inertia weight (*chw*).

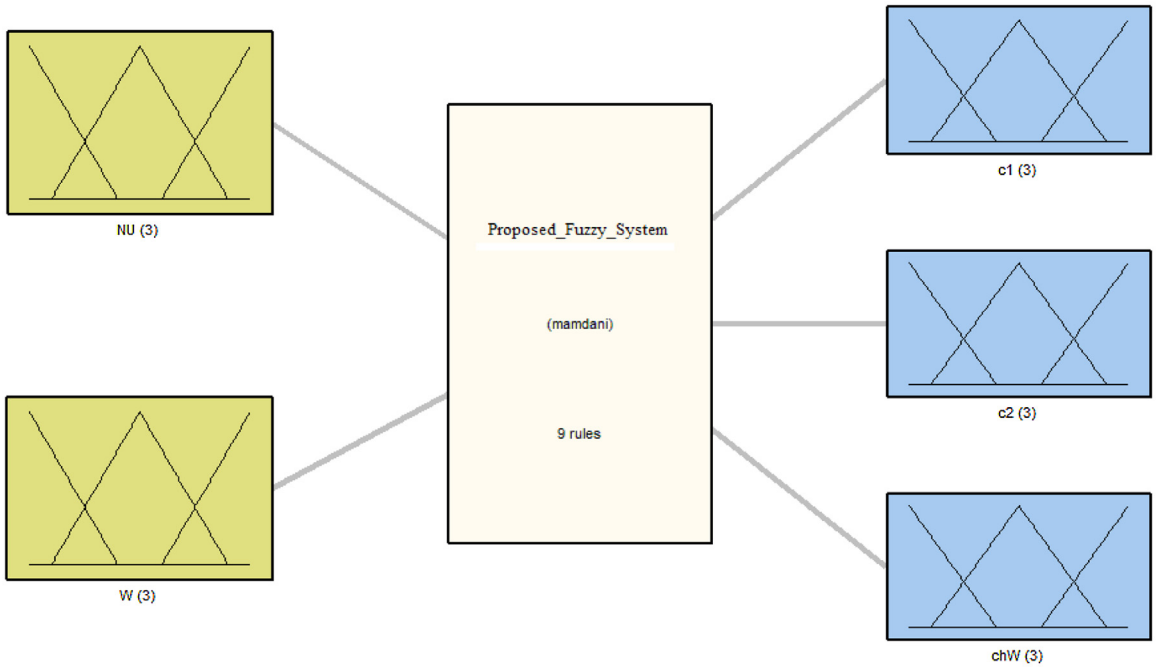


Fig. 3. Proposed fuzzy model.

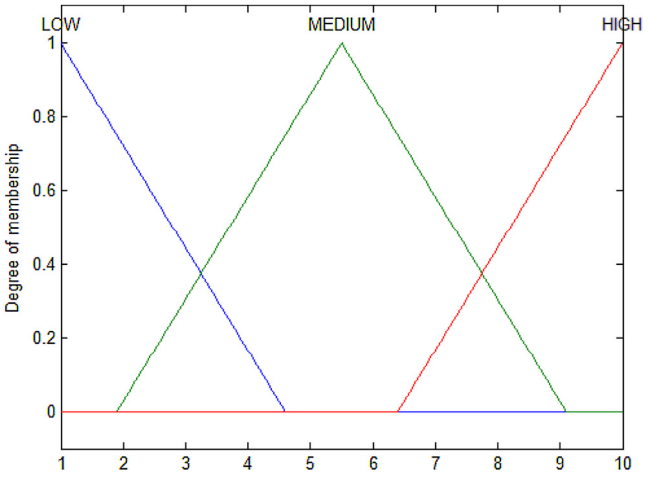


Fig. 4. Membership functions of the proposed fuzzy system.

A Mamdani fuzzy inference system model is used to compute the output variable values. The defuzzification is performed by using the *centroid* of area method:

$$O = \frac{\int_z M_o(z)\, z dz}{\int_z M_o(z)\, dz} \tag{7}$$

The fuzzy inference system structure consists of the following parameters: three linguistic labels, i.e. LOW, MEDIUM and HIGH, are defined for each of the fuzzy variables (*NU*, *w*, c_1, c_2 and *chw*). For the variable *NU*, linguistic labels mean a numerical range of iterations between 1 and 10 values. For the variable *w*, linguistic labels mean a numerical range of values between 0.2 and 1.2. For the variables c_1 and c_2, linguistic labels mean a numerical range of values between 1 and 2. For the variable *chw*, linguistic labels mean a numerical range of values between 0 and 1.

Fig. 4 illustrates the membership functions for each of the fuzzy variables of the proposed fuzzy system.

Table 2 shows the membership function parameters, where we used Triangular membership functions in the proposed fuzzy system. We have found these membership functions to be useful for this work, but other types of membership functions can be used.

We have 9 fuzzy rules from which the change in c_1, c_2 and *chw* are calculated. A sample fuzzy rule is:if normalized *NU* is LOW, and

w value is LOW

then the c_1 is LOW, $c_2$is LOW and *chw* is HIGH

The rules are obtained by experimental knowledge and are indicated in Table 3.

The range of *NU* is [1,10], the value for *w* is bounded in $0.2 \leq w \leq 1.2$ and the values of c_1 and c_2 are bounded in $1 \leq (c_1, c_2) \leq 2$. The mathematical description for c_1 and c_2 is defined with the following expressions

$$c_1 = \frac{\sum_{i=1}^{r_{c_1}} \mu_i^{c_1}(c_{1i})}{\sum_{i=1}^{r_{c_1}} \mu_i^{c_1}} \tag{8}$$

Where: c_1 = Percentage of cognitive acceleration of the particle *i*.
r_{c_1} = Number of rules of the fuzzy system corresponding toc_1.
c_{1i} = Output result for rule *i* corresponding toc_1.
$\mu_i^{c_1}$ = Membership function of rule *i* corresponding toc_1.

$$c_2 = \frac{\sum_{i=1}^{r_{c_2}} \mu_i^{c_2}(c_{2i})}{\sum_{i=1}^{r_{c2}} \mu_i^{c_2}} \tag{9}$$

Where: c_2 = Percentage of cognitive acceleration of particle *i*.
r_{c2} = Number of rules of the fuzzy system corresponding toc_2.
c_{2i} = Output result for rule *i* corresponding toc_2.
$\mu_i^{c_2}$ = Membership function of rule *i* corresponding toc_2.

Table 2
Membership functions parameters.

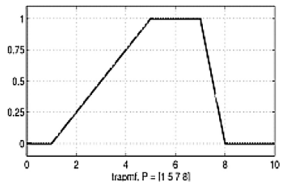
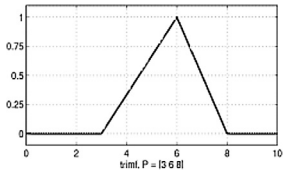
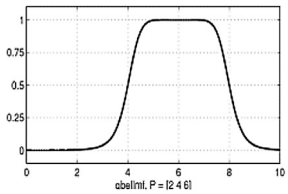
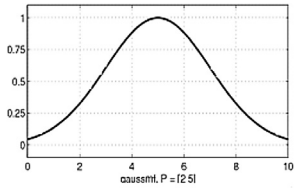
Type of Membership Function	Description	as given by	Shape
Trapezoidal	The trapezoidal curve is a function of a vector, x, and depends on four scalar parameters a, b, c, and d	$f(x;a,b,c,d) = \begin{cases} 0, & x \le a \\ \frac{x-a}{b-a}, & a \le x \le b \\ 1, & b \le x \le c \\ \frac{d-x}{d-c}, & c \le x \le d \\ 0, & d \le x \end{cases}$	
Triangular	The curve is a function of a vector, x, and depends on three scalar parameters a, b, and c,	$f(x;a,b,c) = \begin{cases} 0, & x \le a \\ \frac{x-a}{b-a}, & a \le x \le b \\ \frac{c-x}{c-b}, & b \le x \le c \\ 0, & c \le x \end{cases}$	
The generalized bell	The generalized bell function depends on three parameters a, b, and c	$f(x;a,b,c) = \frac{1}{1+\left\lvert\frac{x-c}{a}\right\rvert^{2b}}$	
Gaussian	The symmetric Gaussian function depends on two parameters σ and c	$f(x;\sigma,c) = e^{\frac{-(x-c)^2}{2\sigma^2}}$	

Table 3
Rules in the fuzzy system for adapting the c_1, c_2 and w of PSO.

Rule number	Input		Output		
	NU	*w*	*chw*	c_1	c_2
1	1	1	3	1	1
2	1	2	1	3	3
3	1	3	1	3	3
4	2	1	3	1	1
5	2	2	2	1	3
6	2	3	1	3	3
7	3	1	3	1	1
8	3	2	1	3	3
9	3	3	1	3	3

5. Co-evolution for PSO

To validate in the best way possible our proposed method we used co-evolution to achieve the best results, this is, because different topologies can be used in the same execution time using different cores of CPUs. Therefore, the method is able to obtain the results with different settings of PSO faster than only using the traditional configuration.

The key motivation to apply the co-evolution concept on high-dimensional space problems is that they can be partitioned into lower dimensional subspaces. In this case in the PSO algorithm we can split the solution vector (fitness) into smaller vectors, with this improving the searching capacity of the method.

Co-evolutionary approaches can be grouped in two categories: competitive and cooperative, and both types of co-evolutionary approaches have shown useful results in solving complex problems. Previous works [9] [24] that have investigated co-evolving multiple cooperative species in divided subpopulations have suggested that a user-specified decomposition of the problem into species is better.

The methodology used in this work is the following:

a.) A subpopulation is represented as a main solution.
b.) The union of representative members of each population represents complete solutions.
c.) The fitness of the complete solutions of participating members determines credit assignment.
d.) The number of subpopulations should itself evolve when needed.
e.) The PSO algorithm handles the evolution of each subpopulation.

A solution to optimize high-dimensional functions consists in decomposing them into N subpopulations each of which contains the main values for a particular parameter. One can then set the fitness to a determined member of an assembled subpopulation, with selected members from other subpopulations to build an N-dimensional vector, where the fitness of a determined member of a particular subpopulation is calculated by estimating how well it "cooperates" with other subpopulations to generate good solutions.

```
Define
v(S,m) = (S_{d+1}.x, ... ,S_n, x)
Create and initialize S_d, d∈ [1 ... n]
Repeat
    for each swarm d = 1, ... , S_n do
        for each particle i = 1, ... , S_n do
            //update the particle's of swarm S_d
            if f(v(S_d,x_i)) < f(v(S_d,y_i)) then
                S_d.y_i = S_d.x_i;
            end
            //update the global best position of swarm S_p
            if f(v(S_d,y_i)) < f(v(S_d,ŷ)) then
                S_d.ŷ = S_d.y_i;
            end
        end
        //update the particle's velocity and position of swarm S_p
        for each particle i = 1, ... , S_n do
            update the particle's velocity, equation (3);
            update the particle's position, equation (2);
        end
    end
t = t + 1
until stop condition, t > max_iteration;
```

Fig. 5. Pseudocode for the PSO algorithm with Co-evolution.

The pseudocode for the PSO algorithm with co-evolution is summarized in Fig. 5. In this algorithm, $S_d.x_i$ denotes the position of particle i in swarm d. The local best position, $S_d.y_i$, associated with particle i is the best position in swarm d the particle has visited since the first time step and $S_d.\hat{y}$ is the global best solution in swarm d. The function v(d,m) returns an n-dimensional vector formed by concatenating all the global best vectors across all the swarms. The parameter m represents the position of any particle form swarm S_d. Then the current best vector is composed of the global best particles $S_d.\hat{y}$ of each of the swarms. Fig. 5 shows the Pseudocode for the PSO algorithm with the co-evolution approach that we used.

6. Optimization experiments

In this section the Particle Swarm Optimization algorithm is implemented for the optimization of 11 benchmark functions commonly known in the literature. All elements of the algorithm, including the simulation parameters and results are described. Co-evolution is used for the high dimensional space functions.

6.1. Parameters of PSO

The functions, the number of dimensions (D) and the optimal solutions are summarized in Table 4. The goal is to reach the optimum of the functions. The maximum iteration number is fixed to 100000. Different swarm sizes of N = 20, 40, 60, 80 and 100 particles are implemented. The Learning cognitive factor(c_1)and Learning social factor (c_2)are static with$c_1 = c_2 = 2$. Values between 0.1 and 0.9 for the Inertia weight (w) were simulated, and therefore the best results are presented. A fully connected topology as explained above was used in all cases. Each optimization experiment was run 30 times with random initial values of x in the range [x_{min},x_{max}].

6.2. Simulation results

In this subsection, simulation results of the functions are presented considering the application of different inertia weight approaches (Constant, random adjustments, linear decreasing, nonlinear decreasing, fuzzy adaptive and the proposed fuzzy system). The "*No exps*" label is the number of experiment sets. In these simulations, we use a different swarm size (N) to find the best size, in which some experiments when "N" is increased the iterations number is decreased. The "*Avg. Iter.*" label shows the average iterations of experiments. The "*Suc. Rat.*" label refers to the success rate that describes the calculation of the number of optimization experiments in which the goal was reached.

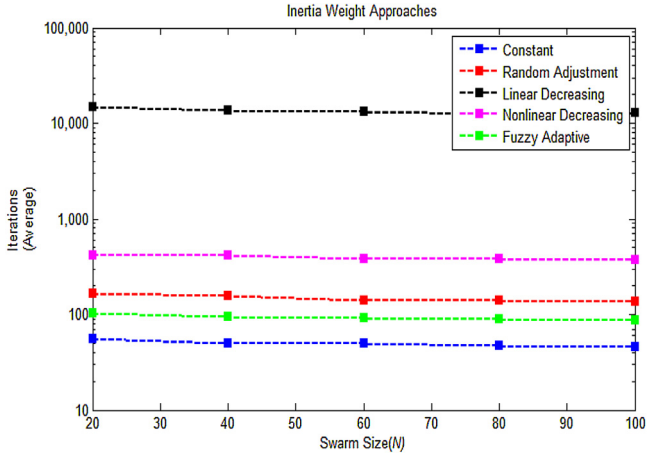


Fig. 6. Information from Table 2 of the F1 function.

6.2.1. F1 function

Table 5 presents simulation results of the F1 function for the different inertia weight approaches. These approaches show their great capacity to find optimal solutions for one-dimensional spaces functions with a full success rate. In this case we can note that these approaches don't have any problems to find the goal solution, therefore it's not necessary to apply our proposed system. This is because in this case we are considering low dimensional spaces.

Fig. 6 illustrates the information from Table 5, where clearly we can observe that the Constant approach with an average iteration of 46.4 is better than the other ones. The value for the Constant approach is 0.1.

6.2.2. F2 function

Table 6 presents simulation results for the F2 function in 2-dimensional spaces. The constant approach gets trapped at least once in a local minimum when a low swarm size is used, such as 20, 40 and 60 with a success rate of 0.66, 0.89 and 0.93 respectively; however other approaches show their good capacity to find the optimal solution for functions in 2-dimensional spaces and therefore the proposed fuzzy system is not necessary.

Fig. 7 illustrates the information from Table 6, where we can clearly observe that the Constant approach with an average iteration of 11.6 is better than the other ones. The value for the Constant approach is 0.1, but this approach it is more susceptible to get trapped in a local minimum.

6.2.3. F3 function

Table 7 presents simulation results for the F3 function in 3-dimensional spaces. All approaches get trapped in local minima and therefore the proposed fuzzy system is implemented with a success rate of 1 in all cases. The best average number of iterations is 5368.2 from our proposed fuzzy system.

Fig. 8 illustrates the information from Table 7, where we can clearly observe that our proposed approach is better than other approaches.

Table 4
General definition for benchmark functions.

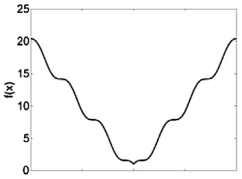
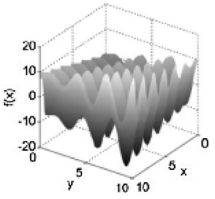
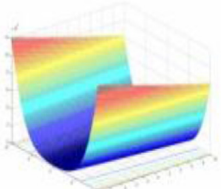
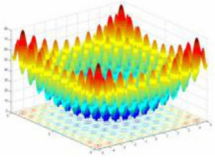
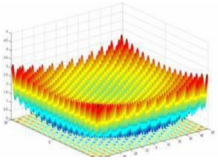
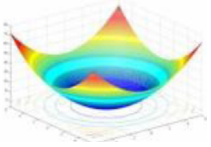
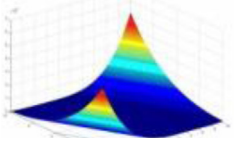
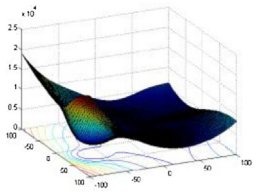
Function	Equation	D	Range $[x_{min}, x_{max}]$	Optimal f	Surface plot
F1	$f_1(x) = \lvert x\rvert + \cos(x)$	1	[−100,100]	1	
F2	$f_2(x) = x_1 \sin(4x_1) + 1.1x_2 \sin(2x_2)$	2	[0,100]	−18.5547	
F3	$f_3(x) = \sum_{i=1}^{N-1} \left\{ 100\left[x_{i+1} - x_i^2\right]^2 + [1 - x_i]^2 \right\}$	N	[−100,100]	0	
F4	$f_4(x) = 10N + \sum_{i=1}^{N} \left[x_i^2 - 10\cos(2\pi x_i)\right]$	N	[−100,100]	0	
F5	$f_5(x) = \sum_{i=1}^{N} \frac{x_i^2}{4000} - \prod_{i=1}^{N} \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$	N	[−100,100]	0	
F6	$f_6(x) = \sum_{i=1}^{N} i * x_i^2$	N	[−100,100]	0	
F7	$f_7(x) = \sum_{i=1}^{N} x_i^2 + \left(\sum_{i=1}^{N} 0.5ix_i\right)^2 + \left(\sum_{i=1}^{N} 0.5ix_i\right)^4$	N	[−100,100]	0	
Summary of the CEC'14 Test Functions (Composition Functions)					
F23	$N = 5$, $\sigma = [10, 20, 30, 40, 50]$ $\lambda = [1, 1e\text{-}6, 1e\text{-}26, 1e\text{-}61e\text{-}6]$ *bias* = [0, |100, 200, 300, 400] g_1: Rotated Rosengborg's Function F_4' g_2: High Conditioned Elliptic Function F_1' g_3: Rotated Bent Cigar Function F_2' g_4: Rotated Discus Function F_3' g_5: High Conditioned Elliptic Function F_1'			2300	

Table 4 (*Continued*)

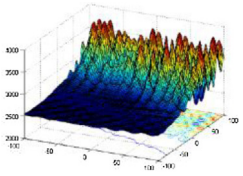
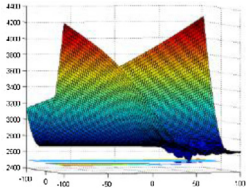
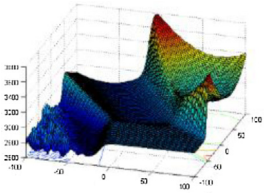
Function	Equation	D	Range $[x_{min}, x_{max}]$	Optimal f	Surface plot
F24	$N = 3$ $\sigma = [20, 20, 20]$ $\lambda = [1, 1, 1]$ *bias* = [0, 100, 200] g_1: Schwefel's Function F_{10}' g_2: Rotated Rastrigin Function F_9' g_3: Rotated HGBat Function F_{14}'			2400	
F25	$N = 3$ $\sigma = [10, 30, 50]$ $\lambda = [0.25, 1, 1e\text{-}7]$ *bias* = [0, 100, 200] g_1: Rotated Schwefel's Function F_{11}' g_2: Rotated Rastrigin Function F_9' g_3: Rotated High Conditioned Elliptic Function F_1'			2500	
F26	$N = 5$ $\sigma = [10, 10, 10, 10, 10]$ $\lambda = [0.25, 1, 1e\text{-}7, 2.5, 10]$ *bias* = [0, 100, 200, 300, 400] g_1: Rotated Schwefel's Function F_{11}' g_2: Rotated HappyCat Function F_{13}' g_3: Rotated High Conditioned Elliptic Function F_1' g_4: Rotated Weierstrass Function F_6' g_5: Rotated Griewank's Function F_7'			2600	

Table 5
Simulation results of the F1 function.

No exps	D	N	Constant		Random Adjustments		Linear Decreasing		Nonlinear Decreasing		Fuzzy Adaptive	
			AvgIter.	Suc.Rat.	Avg.Iter.	Suc.Rat.	Avg. Iter.	Suc.Rat.	Avg. Iter.	Suc.Rat.	Avg. Iter.	Suc.Rat.
1	1	20	56.1	1	163.4	1	14944.3	1	410.4	1	102.3	1
2	1	40	50.3	1	157.5	1	13780.4	1	412.2	1	95.2	1
3	1	60	49.1	1	141.7	1	13252.8	1	380.1	1	90.7	1
4	1	80	47.4	1	140.3	1	**12362.4**	1	383.8	1	89.6	1
5	1	100	**46.4**	1	**137.4**	1	12790.6	1	**375.4**	1	**86.4**	1

Bold means the "best values".

Table 6
Simulation results of the F2 function in 2-dimensional spaces.

No Exps	D	N	Constant[a]		Random Adjustments		Linear Decreasing		Nonlinear Decreasing		Fuzzy Adaptive	
			Avg. Ite.	SucRat.	Avg. Iter.	Suc.Rat.	Avg. Iter.	Suc. Rat.	Avg. Iter.	Suc.Rat.	Avg.Iter.	Suc.Rat.
1	2	20	28.5	0.66	44.1	1	1478.2	1	218.2	1	24.7	1
2	2	40	17.1	0.86	28.6	1	889.7	1	199.3	1	20.7	1
3	2	60	13.8	0.93	29.4	1	624.8	1	184.7	1	19.9	1
4	2	80	12.7	1	27.8	1	538.1	1	181.5	1	17.2	1
5	2	100	**11.6**	1	**23.5**	1	**466.3**	1	**166.2**	1	**15.7**	1

Bold means the "best values".
[a] It gets trapped in a local minimum.

To statistically validate our approach in the best way, a *t* student test was performed. With the aim of demonstrating the difference in run time compared with the methods analyzed in this paper. After applying the test in 3-dimensional spaces, we can observe that our method is significantly faster than some of the other methods because the t values are 2.32, 14.31, 1.25, 0.83 and 0.64 for each approach respectively.

Table 7
Simulation results of the F3 function in 3-dimsenional spaces.

No Exps	D	N	Constant[a]		Random[a] adjustments		Linear[a] decreasing		Nonlinear[a] decreasing		Fuzzy[a] Adaptive		Proposed Fuzzy System	
			Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.
1	3	20	100000	0	100000	0	54373.1	0.33	18410.3	0.36	13787.2	0.30	13583.4	1
2	3	40	18354.7	0.13	17895.8	0.16	49960.2	0.43	11377.9	0.26	11330.7	0.36	11663.7	1
3	3	60	15407.5	0.20	12390.6	0.23	49097.9	0.50	9943.7	0.53	10472.4	0.50	8121.6	1
4	3	80	12371.6	0.33	13051.6	0.26	43678.3	0.56	8528.6	0.70	8189.3	0.76	6476.3	1
5	3	100	**11751.3**	0.53	**10989.9**	0.43	**42135.4**	0.66	**7262.3**	0.73	**7564.4**	0.73	**5368.2**	1

Bold means the "best values".
[a] It gets trapped in a local minimum.

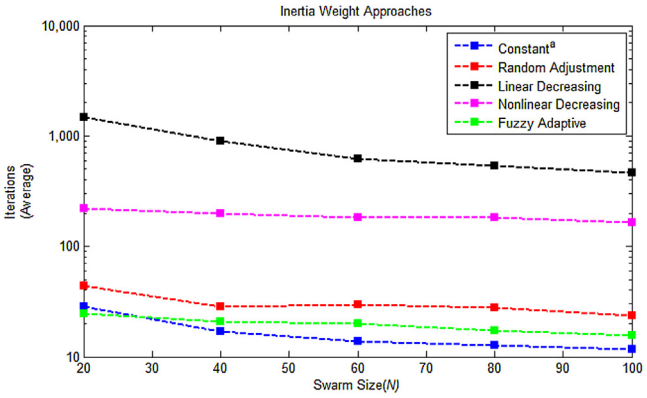


Fig. 7. Information from Table 6 of the F2 function in 2-dimensional spaces.

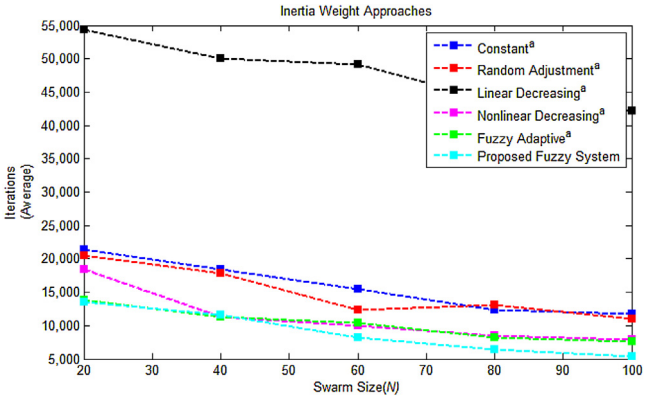


Fig. 8. Information from Table 7 of the F3 function in 3-dimensional spaces.

Table 8 presents simulation results for the F3 function in 8-dimensional spaces. All approaches get trapped in local minima and therefore the proposed fuzzy system is implemented with a success rate of 1 in all cases. Our proposed fuzzy system is the best approach to get closer to the optimum with a minimum of **2.885e-28** without getting trapped.

Fig. 9 shows information from Table 8, where we can clearly observe that our proposed approach is better than other approaches.

After applying the t student test in 8-dimensional spaces, the t values are 2.50, 2.43, 7.26, 2.14 and 5.81 for each approach, respectively. This means that the proposed method outperforms all the other approaches.

6.2.4. F4 function

Table 9 presents simulation results for the F4 function in 8-dimensional spaces. All approaches get trapped in local minima and therefore the proposed fuzzy system is implemented with a

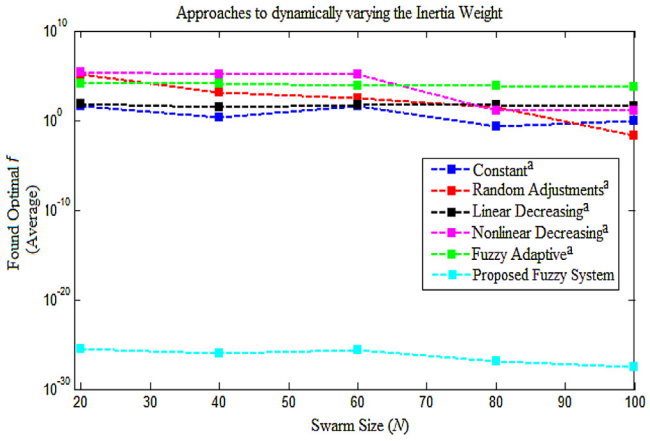


Fig. 9. Information from Table 8 of the F3 function in 8-dimensional spaces.

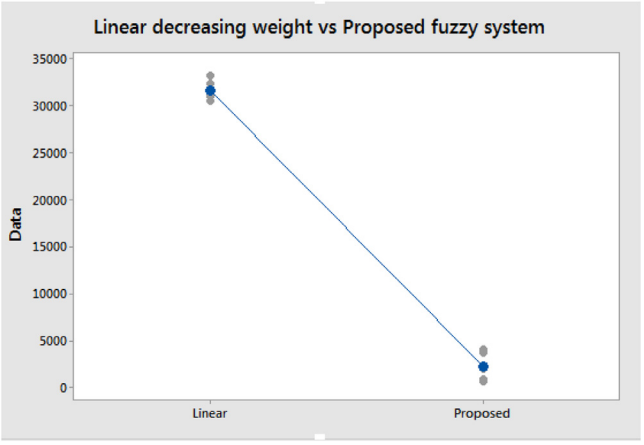


Fig. 10. T Student Test.

success rate of 1 in all cases. The best average number of iterations is 673.5 is from our proposed fuzzy system.

After applying the test in 8-dimensional spaces, the t values are 2.24, 4.28, 34.40, 7.47 and 4.24 for each approach respectively. Table 10 presents simulation results for the F4 function. All approaches get trapped in local minima from eight-dimensional space onwards therefore the proposed approach is implemented to high-dimensional spaces, such as 10, 20 and 30. The best average number of iterations is 1935.6, 5983.7 and 17492.7 respectively in 10, 20 and 30 dimensional spaces.

To validate in the best form possible our proposed method we perform a t student test between the linear decreasing weight and the proposed fuzzy system to adapt the weight parameter in PSO in the F4 function. The number of data used were 5, in this case, the

Table 8
Simulation results of the F3 function in 8-dimensional spaces.

No exps	D	N	Constant[a]		Random[a] adjustments		Linear[a] decreasing		Nonlinear[a] decreasing		Fuzzy[a] adaptive		Proposed Fuzzy System	
			Found Optimal f	Suc Rat	Found Optimal f	Suc. Rat.	Found Optimal f	Suc. Rat.	Found Optimal f	Suc. Rat.	Found Optimal f	Suc Rat	Found Optimal f	Suc. Rat.
1	8	20	44.2982	0	125145.97	0	75.6	0	250047.1	0	13583.4	0	3.025e-26	1
2	8	40	1.9093	0.16	1273.59	0	32.6	0	126306.5	0	11663.7	0	8.659e-27	1
3	8	60	44.2186	0	290.32	0	54.1	0	125167.4	0	8121.6	0	2.168e-26	1
4	8	80	0.2101	0.26	32.445	0	53.2	0	11.8	0	6476.3	0	1.373e-27	1
5	8	100	0.7971	0.33	0.0178	0.16	43.2	0	**10.8**	0	**5368.2**	0	**2.885e-28**	1

Bold means the "best values".
[a] It gets trapped in a local minimum.

average of iterations that was necessary for find the global minimum. We can notice in Fig. 10 that the difference is very significant. The t value is of 34.40 and the standard deviation for the linear decreasing weight was 1083, and for the proposed fuzzy system was 1574. This test was made with a degree of reliability of 95%.

Table 11 shows simulation results for the F4 function in 8-dimensional spaces. In this case the swarm size is increased to 200 and 300 to observe the behavior of the algorithm. We can notice a better performance from both approaches with 587.9 and 511.3 respectively for the Constant and Proposed approaches, but the constant approach gets trapped in a local minimum.

The Co-evolution concept is also applied, where we can note the constant approach is better than the proposed approach without getting trapped in local minima. In this work the proposed fuzzy system is applied to avoid getting trapped on high-dimensional spaces, but with the Co-evolution concept we can note that the Constant approach does not have problems of getting trapped, this because the Co-evolution concept simplifies the problem.

Fig. 11 shows the information from Table 11 where we can notice that the Constant approach with the Co-evolution concept is better than the proposed approach.

After applying the test for the Constant approach in 8-dimensional space with the Co-evolution concept over the Constant approach without the Co-evolution concept, the t value is 2.36 and the proposed fuzzy system in 8-dimensional with the Co-evolution concept over the proposed fuzzy system without the Co-evolution concept, the t value is 9.65. When the dimensional spaces are increased in complexity in any function, the proposed fuzzy system is significantly better than the other methods.

Table 12 shows the simulation results for the F4 function in 30-dimensional spaces. We can notice that in previous experiments the best swarm size is 300 so we used this one with 30-dimensional spaces. The Constant approach in each experiment got trapped, while the proposed approach found the optimal solution without falling into a local minimum. Applying the Co-evolution concept we can note that both the Constant and proposed fuzzy they are able to find the optimal solution.

6.2.5. F5, F6 and F7 functions

Table 13 shows the optimization results of different functions, namely F6, F7 and F8, in 30-dimensional spaces. The best approach (Proposed fuzzy system) without getting trapped in a local minimum on high-dimensional spaces with the best swarm size (300) is applied. In these cases we measure the time (HH:MM:SS: mils) to compare with the Co-evolution concept.

Fig. 12 shows comparative results of the proposed fuzzy system approach with the Co-evolution concept in the F4, F5, F6 and F7 functions. We can note that co-evolution reduces the number of iterations with proposed approach.

In the Wilcoxon test (Fig. 13) we show that the results obtained by the proposed co-evolution method were significantly better than the proposed fuzzy method without co-evolution. Because in the Wilcoxon test, an obtained W is significant if it is less than or equal to the critical value, in this case the values were W (0) = 1, p > 0.1, two tailed test.

6.2.6. F23, F24, F25 and F26 functions

Table 14 shows the optimization results of different functions, namely F23, F24 and F25, in 2-dimensional spaces. The best approach without getting trapped in a local minimum is the proposed fuzzy system.

Table 14 shows the optimization results of different functions, namely F23, F24, F25 and F26, in 10-dimensional spaces. Each algorithm is trapped in a local minimum with 10000 epochs (Table 15).

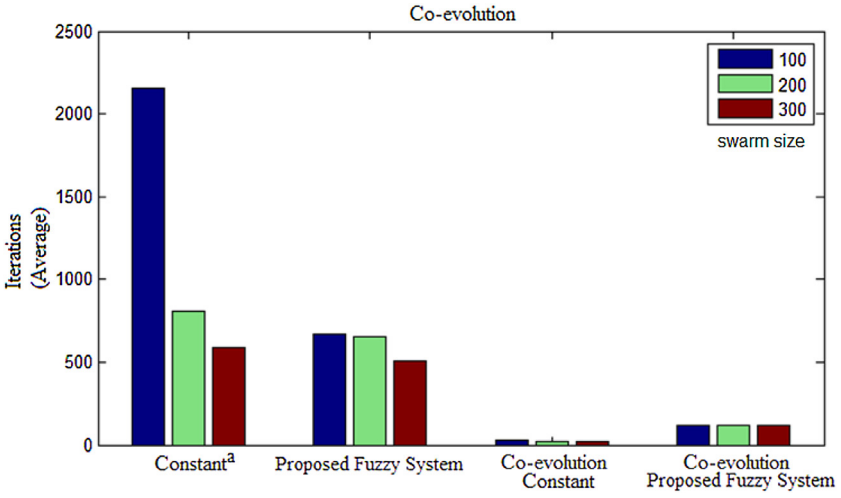


Fig. 11. Information from Table 11 of the F4 function in 8-dimensional spaces.

Table 9
Simulation results of the F4 function in 8-dimensional spaces.

No exps	D	N	Constant[a]		Random[a] adjustments		Linear[a] decreasing		Nonlinear[a] decreasing		Fuzzy[a] Adaptive		Proposed Fuzzy System	
			Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.
1	8	20	100000	0	100000	0	33227.8	0.40	100000	0.16	100000	0.2	4064.6	1
2	8	40	100000	0	75560.4	0.10	32305.3	0.46	100000	0.13	100000	0.26	3740.1	1
3	8	60	100000	0.06	50465.3	0.23	31269.1	0.30	100000	0.20	100000	0.23	2023.4	1
4	8	80	6534.4	0.40	27040.1	0.33	30979.4	0.40	62773.5	0.30	100000	0.33	883.7	1
5	8	100	**2151.8**	0.56	**39308.5**	0.50	**30566.4**	0.76	**50837.9**	0.26	**6843.9**	0.83	**673.5**	1

Bold means the “best values”.
[a] It gets trapped in a local minimum.

Table 10
Simulation results of the F4 function in 10, 20 and 30 dimensional spaces.

No exps	D	N	Proposed Fuzzy System	
			Avg. Inter.	Suc Rat.
1	10	20	5623.7	1
2	10	40	4018.5	1
3	10	60	2761.3	1
4	10	80	1955.4	1
5	10	100	**1935.6**	1
1	20	20	45642.6	1
2	20	40	38206.4	1
3	20	60	18821.3	1
4	20	80	10017.6	1
5	20	100	**5983.7**	1
1	30	20	52671.6	0.93
2	30	40	39357.7	0.96
3	30	60	28866.2	1
4	30	80	22006.3	1
5	30	100	**17492.7**	1

Bold means the “best values”.

Table 11
Simulation results of the F4 function in 8-dimensional spaces with the Co-evolution concept.

No exps	D	N			Co-evolution	
			Constant[a] Avg. Iter.	Proposed Fuzzy System Avg. Iter.	Constant Avg. Iter.	Proposed Fuzzy System Avg. Iter.
1	8	100	2151.8	673.5	26.2	117
2	8	200	803.7	654.8	24.1	121
3	8	300	**587.9**	**511.3**	**23.2**	**118**

Bold means the “best values”.
[a] It gets trapped in a local minimum.

Table 12
Simulation results of the F4 function in 30-dimensional spaces with the Co-evolution concept.

No Exps	D	N			Co-evolution	
			Constant[a]	Proposed Fuzzy System	Constant	Proposed Fuzzy System
			Avg. Iter.	Avg. Iter.	Avg. Iter.	Avg. Iter.
1	30	300	–	9641.5	23.2	121.1

–Each experiment got trapped.
[a] It gets trapped in a local minimum.

Table 13
Proposed Fuzzy System Performance using the Co-evolution concept.

No Exps	Fn	D	N			Co-evolution			
				Proposed Fuzzy System		Proposed Fuzzy System		Constant	
				Avg. Iter.	Average Time	Avg. Iter.	Average Time	Avg. Iter.	Average Time
1	F5	30	300	902.3	00:05:11:365	115.7	00:00:08:987	23.3	00:00:00:911
1	F6	30	300	21051.8	00:58:19:369	3226.7	00:01:55:134	598.1	00:00:20:541
1	F7	30	300	62404.9	05:17:23:013	3505.3	00:04:50:092	620.9	00:00:25:862

Table 14
Simulation results of the F23, F24, F25 and F26 functions in 2-dimensional spaces.

Fn	D	N	Constant[a]		Random[a] adjustments		Linear[a] decreasing		Nonlinear[a] decreasing		Fuzzy[a] Adaptive		Proposed Fuzzy System	
			Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.
F23	2	100	856	2300	234	2300	2141	2300	143	2300	160	2300	213	2300
F24	2	100	896	2400	221	2400	2029	2400	70	2400	174	2400	237	2400
F25	2	100	725	2500	321	2500	1978	2500	61	2500	170	2500	211	2500
F26	2	100	1210	2600	247	2600	2051	2600	89	2600	213	2600	245	2600

[a] It gets trapped in a local minimum.

Table 15
Simulation results of the F4 function in 8-dimensional spaces.

Fn	D	N	Constant[a]		Random[a] adjustments		Linear[a] decreasing		Nonlinear[a] decreasing		Fuzzy[a] Adaptive		Proposed Fuzzy[a] System	
			Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.	Avg. Iter.	Suc Rat.
F23	10	100	10000	2629.4	10000	2629.4	10000	2629.4	10000	2637.6	10000	2629.4	10000	2619.8
F24	10	100	10000	2518.5	10000	2518.2	10000	2532.4	10000	2538.5	10000	2517.3	10000	2519.8
F25	10	100	10000	2704.8	10000	2702.3	10000	2702.3	10000	2701.9	10000	2701.5	10000	2700.1
F26	10	100	10000	2700.1	10000	2701.5	10000	2701.5	10000	2700.2	10000	2701.2	10000	2700.1

[a] It gets trapped in a local minimum.

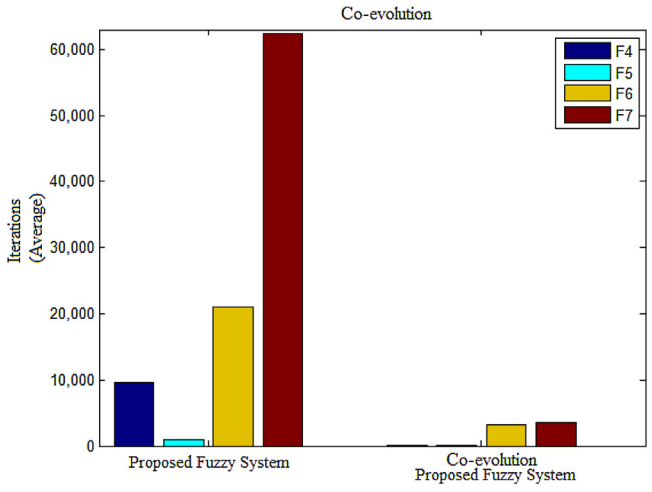


Fig. 12. Comparative results for the functions F4, F5, F6 and F7.

7. Conclusions

In this paper, a comparative study of inertia weight parameter adaptation strategies is realized. Such approaches to inertia weight adaptation are: constant, random adjustments, linear decreasing, nonlinear decreasing and fuzzy adaptive inertia. These approaches easily get trapped in the local optima in the optimization of high-dimensional space functions. Based on such results, a fuzzy particle swarm optimization is proposed. The fuzzy proposed system avoids getting trapped in the local optima and it improves the performance of the PSO algorithm. It consists of two inputs and three outputs to obtain the new inertia weights, c_1 and c_2. For problems of high-dimensional functions, the concept co-evolution is also applied. As shown in the benchmark functions tests, the co-evolution concept in terms of time of the Central Processing Unit (CPU) shows to be an efficient approach, an improved convergence speed needed to reach suitable solutions for benchmark functions showing better results in this work. Simulation tests on eleven mathematical functions were performed to compare the proposed method. Simulation

Hip. Nula H_0 :	F1 = F2
Hip. Alterna H_a :	F1 ≠ F2

Function	Proposed Fuzzy System	Co-evolution Proposed Fuzzy System	Diference	Absolute Difference	Ranked	signo	Range (-)	Range(+)
F4 (8-Dimensional)	613.20	118.67	494.53	494.53	1	1		1
F4 (30-Dimensional)	9641.5	121.1	9520.4	9520.4	3	1		3
F5	902.3	115.7	786.6	786.6	2	1		2
F6	21051.8	3226.7	17825.1	17825.1	4	1		4
F7	62404.9	3505.3	58899.6	58899.6	5	1		5

		$W^- =$	0
		$W^+ =$	15
		$W =$	0
	significance level	$p =$	0.1
		$N=$	5
	critical value	$W_0 = W_{a,m} =$	**1**

Fig. 13. Wilcoxon test.

results shows that when more variables are considered there is a higher possibility of getting trapped in a local optimum, therefore, the proposed fuzzy system in the PSO shows better results avoiding getting trapped. Parameters of the proposed fuzzy system can be adjusted (input variables, type of membership and rules) to improve even further the results and this can be future work.

References

[1] A. Carlisle, G. Dozier, Adapting Particle Swarm Optimization to Dynamic Environments. PhD Thesis, Auburn University, 2002.
[2] A. Mitchell Potter, A. De Jong, A Cooperative Coevolutionary Approach to Function Optimization, Computer Science Department, George Mason University, USA, 1994.
[3] C. Russell, Eberthart, Hu Xiaohui, Human Tremor Analysis Using Particle Swarm Optimization, Purdue School of Engineering and Technology, Indiana University Purdue University, Indianapolis, 1999.
[4] F. Valdez, P. Melin, O. Castillo, Evolutionary method combining particle swarm optimization and genetic algorithms using fuzzy logic for decision making, Proceedings of the IEEE International Conference on Fuzzy Systems (2009) 2114–2119.
[5] F. Valdez, P. Melin, O. Castillo, An improved evolutionary method with fuzzy logic for combining Particle Swarm Optimization and Genetic Algorithms, Appl. Soft Comput. 11 (2) (2011) 2625–2632.
[6] H. Shaheen, Voltage Control Based on Fuzzy Adaptive Particle Swarm Optimization, Lambert Academic Publishing, 2011.
[7] H. Yoshida, Y. Fukuyama, S. Takayuma, Y. Nakanishi, A particle swarm optimization for reactive power and voltage control in electric power systems considering voltage security assessment, Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics vol. 6 (October) (1999) 497–502.
[8] J.C. Bansal, P.K. Singh, Mukesh Saraswat, Abhishek Verma, Shimpi Singh Jadon, Ajith Abraham, Inertia weight strategies in particle swarm optimization, in: Proceeding Of: Third World Congress on Nature & Biologically Inspired Computing, NaBIC 2011, Salamanca, Spain, October 19–21, 2011, pp. 633–640.
[9] J. Kennedy, W. Spears, Matching algorithms to problems: an experimental test of the particle swarm and some genetic algorithms on the multimodal problem generator, in: Proceedings of the IEEE Congress on Evolutionary Computation, IEEE Press, 1988, pp. 78–83.
[10] J. Kennedy, The behaviour of particles, Evol. Progr. VII (1998) 581–587.
[12] J. Kennedy, R.C. Eberhart, Particle swarm optimization, in: Proceedings of the IEEE International Joint Conference on Neuronal Networks, IEEE Press, 1995, pp. 1942–1948.
[13] J. Kennedy, Small worlds and megaminds: effects of neighborhood topology on particle swarm performance, in: Proceedings of IEEE Congress on Evolutionary Computation, Washington D.C., USA, 1999, pp. 1931–1938.
[14] J. Peng, Y. Chen, R.C. Eberhart, Battery pack state of charge estimator design using computational intelligence approaches, Proceedings of the Annual Battery Conference on Applications and Advances (2000) 173–177.
[15] J. Salerno, Using the particle swarm optimization technique to train a recurrent neural model, in: Proceedings of the IEEE International Conference on Tools with Artificial Intelligence, IEEE Press, 1997, pp. 45–49.
[17] M. Clerc, The swarm and the queen: towards a deterministic and adaptive particle swarm optimization, Proceedings of the IEEE Congress on Evolutionary Computation vol. 3 (1999) 1951–1957.
[18] M. Clerc, J. Kennedy, The particle swarm-Explosion, stability, and convergence in a multidimensional complex space, IEEE Trans. Evol. Comput. 6 (1) (2002) 58–73.
[19] M.F. Tasgetiren, Y.C. Liang, M. Sevkli, G. Gencyilmaz, A particle swarm optimization algorithm for makespan and total flowtime minimization in the permutation flowshop sequencing problem, Eur. J. Oper. Res. 177 (2007) 1930–1947.
[20] P. Engelbrecht, Frans van den Bergh, A cooperative approach to particle swarm optimization, IEEE Trans. Evol. Comput. 8 (3) (2004).
[21] P.F. Ribeiro, W. Kyle Schlansker, A hybrid particle swarm and neuronal network approach for reactive power control, IEEE (2006).
[22] P.J. Angeline, Evolutionary optimization versus particle swarm optimization: philosophy and performance differences, Proceeding of the Seventh Annual Conference on Evolutionary Programming (1998) 601–610.
[23] R. Brits, A.P. Engelbrecht, F. van den Bergh, A niching particle swarm optimizer, Proceedings of the Fourh Asia-Pacific Conference on Simulated Evolution and Learning (2002) 692–696.
[24] R.C. Eberhart, Y. Shi, Particle swarm optimization: developments, applications and resources, IEEE Press, in: Proceedings of the IEEE Congress on Evolutionary Computation, vol. 1, 2001, pp. 27–30.
[25] R.C. Eberhart, J. Kennedy, A new optimizer using particle swarm theory, Procedings of the Sixth International Symposium on MicroMachine and Human Science (1995) 39–43.
[26] R.C. Eberhart, Y. Shi, Comparing inertia weights and constriction factors in particle swarm optimization, Proceedings of the IEEE Congress on Evolutionary Computation vol. 1 (2000) 84–88.
[27] R. Paul Wiegand, An Analysis of Cooperative Coevolutionary Algorithms, University North Carolina, Charlotte, 1999.
[28] S. Naka, T. Genji, T. Yura, Y. Fukuyama, Practical distribution state estimation using hybrid particle swarm optimization, IEEE Power Engineering Society Winter Meeting vol. 2 (2001) 815–820.
[30] T.I. Cristian, The particle swarm optimization algorithm: convergence analysis and parameter selection, Inf. Process. Lett. 85 (6) (2003) 317–325.
[31] Juan Carlos Vazquez, Fevrier Valdez, Patricia Melin, Comparative study of particle swarm optimization variants in complex mathematics functions, fuzzy logic augmentation of nature-Inspired optimization metaheuristics Studies in Computational Intelligence, vol. 574, 2015, pp. 163–178, Sep 2014.
[32] W. Zhang, Y. Liu, multi-objective reactive power and voltage control based on fuzzy optimization strategy and fuzzy adaptive particle swarm, Electr. Power Energy Syst (2008).
[34] Y. Noa Vargas, Estrategias para mejorar el Balance entre Exploración y Explotación en Optimización de Enjambre de Partículas, Universidad de la Habana, Facultad de Matemáticas y Computación, Diciembre de, 2011.
[35] Y.H. Shi, R.C. Eberhart, Fuzzy adaptive particle swarm optimization, IEEE Int. Conf. on Evolutionary Computation (2001) 101–106.
[36] Y.H. Shi, R.C. Eberhart, Parameter Selection in Particle Swarm Optimization, in: V.W. Porto, D. Waagen (Eds.), EP 1998. LNCS, vol. 1447, Springer, Heidelberg, 1998, pp. 591–600.
[37] Y.H. Shi, R.C. Eberhart, A modified particle swarm optimizer, Proceedings of the IEEE International Conference on Evolutionary Computation (1998) 69–73.
[38] Y.H. Shi, R.C. Eberhart, Fuzzy adaptive particle swarm optimization, IEEE Press, in: Proceedings of the IEEE Congress on Evolutionary Computation, vol. 1, 2001, pp. 101–106.
[39] Z. Zhan, Jun Zhang, Adaptive Particle Swarm Optimization, vol. 5217, Department of Computer Science, Sun Yat-sen University, China, 2008, pp. 227–234.
[40] Oscar Castillo, Patricia Ochoa, José Soria, Differential evolution with fuzzy logic for dynamic adaptation of parameters in mathematical function optimization, Imprecision and Uncertainty in Information Representation and Processing (2016) 361–374.

[41] Cinthya Solano-Aragón, Oscar Castillo, Fuzzy logic augmentation of nature-inspired optimization metaheuristics, in: Optimization of Benchmark Mathematical Functions Using the Firefly Algorithm with Dynamic Parameters, 2015, pp. 81–89.
[42] Emer Bernal, Oscar Castillo, José Soria, Imperialist competitive algorithm applied to the optimization of mathematical functions: a parameter variation study, in: Design of Intelligent Systems Based on Fuzzy Logic, Neural Networks and Nature-Inspired Optimization, 2015, pp. 219–232.
[43] Maribel Guerrero, Oscar Castillo, Mario García Valdez, Cuckoo search algorithm via Lévy flight with dynamic adaptation of parameter using fuzzy logic for benchmark mathematical functions, in: Design of Intelligent Systems Based on Fuzzy Logic, Neural Networks and Nature-Inspired Optimization, 2015, pp. 555–571.